Solution: Greatest Common Divisor (GCD)

Problem Statement

Write recursive code to calculate the Greatest Common Divisor (GCD) of Two Positive Numbers.

The greatest common divisor (GCD) of two positive integers A and B is the largest positive integer that divides both A and B without leaving a remainder.

Let's see some example inputs/outputs for this example:

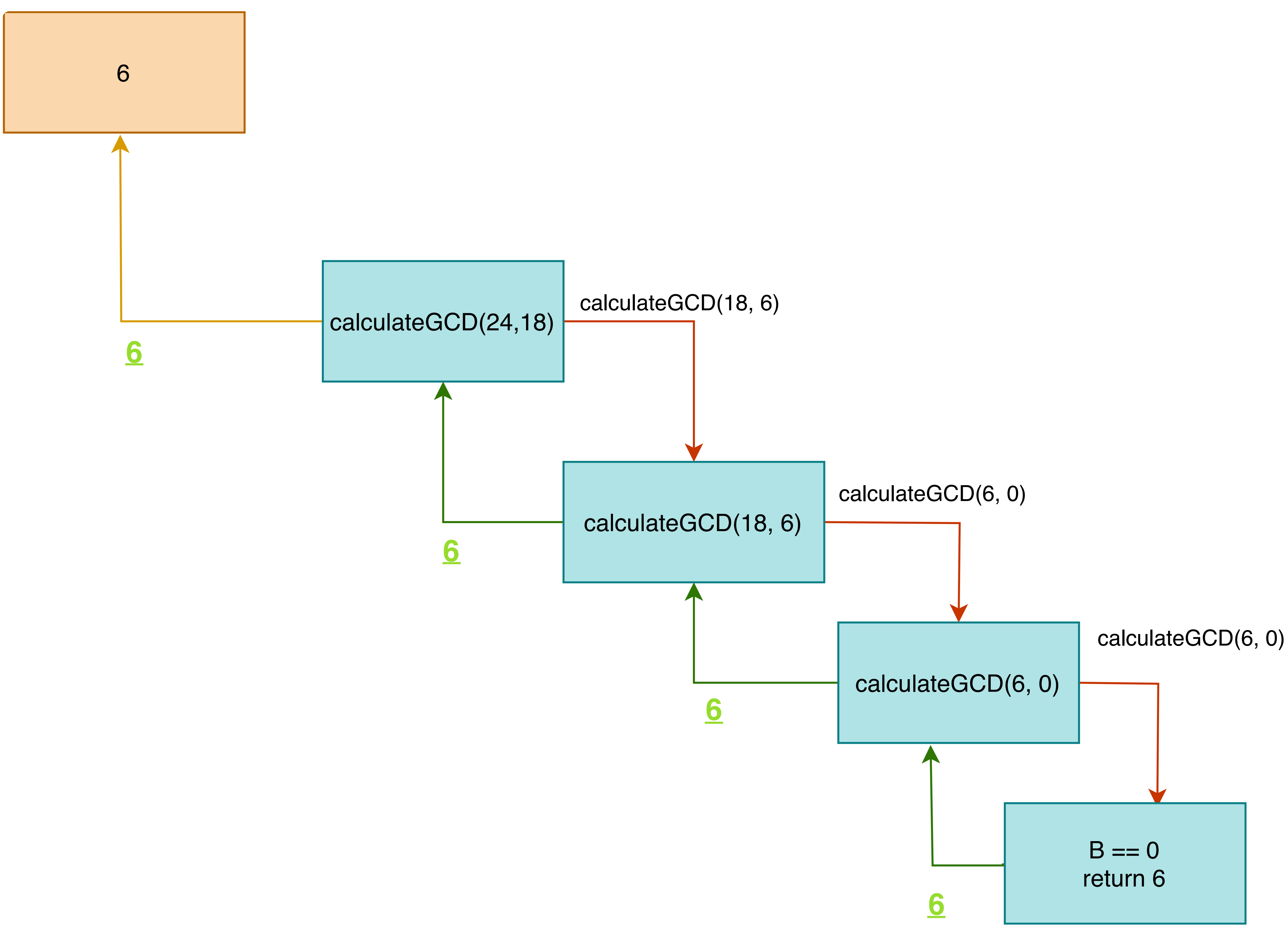
Input(s)	Output(s)	Explanation
A = 12, B = 18	GCD = 6	The factors of 12 are [1, 2, 3, 4, 6, 12], and the factors of 18 are [1, 2, 3, 6, 9, 18]. The common factors between 12 and 18 are [1, 2, 3, 6], and the largest common factor is 6.
A = 25, B = 15	GCD = 5	The factors of 25 are [1, 5, 25], and the factors of 15 are [1, 3, 5, 15]. The common factors between 25 and 15 are [1, 5], and the largest common factor is 5.
A = 40, B = 60	GCD = 20	The factors of 40 are [1, 2, 4, 5, 8, 10, 20, 40], and the factors of 60 are [1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60]. The common factors between 40 and 60 are [1, 2, 4, 5, 10, 20], and the largest common factor is 20.

Solution

The algorithm to calculate the GCD of two positive numbers using recursion can be defined as follows:

1. If B is equal to 0, return A as the GCD (base case).
2. Otherwise, return the recursive call to calculate the GCD of B and the remainder of A divided by B.

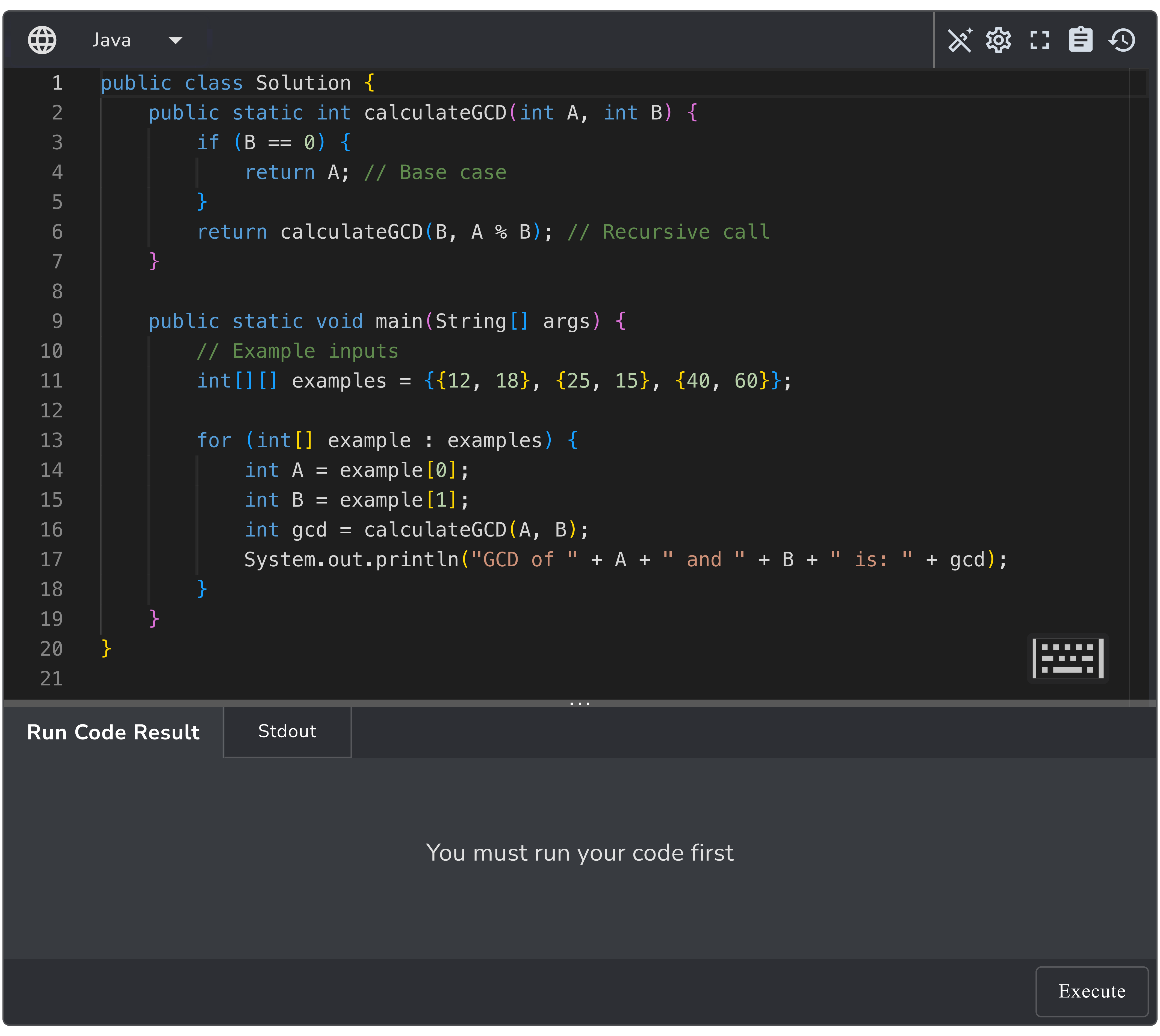
The recursive approach works by utilizing the property that the GCD of two numbers remains the same if we replace the larger number with the remainder of its division by the smaller number. This property is known as the Euclidean algorithm. The algorithm continuously reduces the problem to a smaller instance until the base case is reached, where B is 0. At this point, A represents the GCD of the original numbers.



Example dry run to calculate GCD of 24 and 18

Code

Here is the code for this algorithm:



```java
public class Solution {
    public static int calculateGCD(int A, int B) {
        if (B == 0) {
            return A; // Base case
        }
        return calculateGCD(B, A % B); // Recursive call
    }

    public static void main(String[] args) {
        // Example inputs
        int[][] examples = {{12, 18}, {25, 15}, {40, 60}};

        for (int[] example : examples) {
            int A = example[0];
            int B = example[1];
            int gcd = calculateGCD(A, B);
            System.out.println("GCD of " + A + " and " + B + " is: " + gcd);
        }
    }
}
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(log(min(A, B)))$ since the algorithm reduces the problem size by taking the remainder of A divided by B, and the numbers are divided approximately in half in each recursive call.

The space complexity is $O(log(min(A, B)))$ as well, as the maximum depth of the recursion depends on the minimum value between A and B.

← Previous
3. Greatest Common Divisor (GCD)

Next →
4. Converting Decimal To Binary

☐ Mark as Completed